Studio Nx

JS 260 — 붙이기 · 고치기 · 떼기 · 순서

2026-10-18

JavaScript 260 — 붙이기 · 고치기 · 떼기 · 순서 (`addDevice` · `tailSlot` · `updateDevice` · `removeDevice` · `moveEffect` · `byId`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-09 악기·이펙트 화면: 장치를 붙이고 고치고 떼고 순서를 바꿉니다 (FR-244 · FR-245 · FR-263)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	259 플러그인 고르기 창 · 261 상세 설정 모달

1. 하는 일

네 함수가 서버의 장치 줄을 다룹니다. `addDevice` 는 고른 플러그인을 트랙에 붙이고(악기는 instruments, 이펙트는 plugins) 기본값으로 시작합니다. `tailSlot` 은 붙일 자리를 정합니다. `updateDevice` 는 값 · 바이패스 · 솔로 · 볼륨 · 자리 하나를 고칩니다. `removeDevice` 는 확인을 받고 뗍니다. `moveEffect` 는 이펙트 차례를 바꾸는데, 이펙트 칸 번호에는 `UQ(track_id, slot_index)` 라는 중복 금지 규칙이 있어 옮길 카드를 빈 칸 100 에 잠깐 두고 사이 카드를 한 칸씩 밀거나 당깁니다.

```
moveEffect(track, 1, 0)
  ① [A][B] 에서 B 를 slot 100 으로 비켜 둠   →  A(0), B(100)
  ② 사이 카드를 한 칸씩 당김                →  A(1), B(100)
  ③ B 를 갈 자리로                          →  A(1), B(0)
```

2. 언제 쓰나

쓰는 곳	
`wireCells` (263) · 고르기 창 콜백	`addDevice` · `removeDevice` 를 부릅니다
`wireChainDrag` (263)	끌어 놓은 자리로 `moveEffect` 를 부릅니다
`wireControls` (258) · `saveDeviceModal` (261) · `toggleBypass` (262)	`updateDevice` 를 부릅니다

3. 코드

```
export async function addDevice(track, kind, plugin) {
  const d = devices[track.track_id] || { instruments: [], effects: [] };
  devices[track.track_id] = d;
  const list = kind === 'INSTRUMENT' ? d.instruments : d.effects;
  const body = { plugin_id: plugin.plugin_id, slot_index: tailSlot(list), params: defaultParams(plugin)
};
  try {
    const path = kind === 'INSTRUMENT'
      ? '/tracks/' + track.track_id + '/instruments'
      : '/tracks/' + track.track_id + '/plugins';
    const saved = await api.post(path, body);
    list.push(saved);
```

```
    refreshCells(track);
    showToast((kind === 'INSTRUMENT' ? 'Instrument' : 'Effect') + ' added: ' + plugin.name);
  } catch (err) {
    warn(err);
  }
}

/** 새 장치를 붙일 자리 (FR-245) — 지금 것들 뒤(가장 큰 slot_index + 1)가 첫 후보입니다. */
function tailSlot(list) {
  const used = {};
  let top = -1;
  list.forEach(function (x) {
    used[x.slot_index] = true;
    if (x.slot_index > top) top = x.slot_index;
  });
  if (top + 1 <= 126 && !used[top + 1]) return top + 1;
  let n = 0;
  while (used[n] && n < 127) n += 1;
  return n;
}

export async function updateDevice(track, kind, device, patch) {
  try {
    const path = kind === 'INSTRUMENT'
      ? '/tracks/' + track.track_id + '/instruments/' + device.track_instrument_id
      : '/tracks/' + track.track_id + '/plugins/' + device.track_plugin_id;
    const saved = await api.patch(path, patch);
    Object.assign(device, saved);
    return saved;
  } catch (err) {
    warn(err);
    return null;
  }
}

export async function removeDevice(track, kind, device) {
  const plugin = pluginById(device.plugin_id);
  const name = plugin ? plugin.name : 'this device';
  if (window.confirm && !window.confirm('Remove ' + name + '?')) return;
  try {
    const path = kind === 'INSTRUMENT'
      ? '/tracks/' + track.track_id + '/instruments/' + device.track_instrument_id
      : '/tracks/' + track.track_id + '/plugins/' + device.track_plugin_id;
    await api.delete(path);
    const d = devices[track.track_id];
    if (d) {
      const list = kind === 'INSTRUMENT' ? d.instruments : d.effects;
      const at = list.indexOf(device);
      if (at >= 0) list.splice(at, 1);
    }
    refreshCells(track);
    if (modal && modal.device === device) closeDeviceModal();
    showToast(name + ' removed');
  } catch (err) {
    warn(err);
  }
}
```

```
export async function moveEffect(track, from, to) {
  const d = devices[track.track_id];
  if (!d) return;
  const list = sorted(d.effects);
  if (from === to || from < 0 || to < 0 || from >= list.length || to >= list.length) return;
  const ids = list.map(function (x) { return x.track_plugin_id; });
  const movedId = ids.splice(from, 1)[0];
  ids.splice(to, 0, movedId);
  try {
    await updateDevice(track, 'EFFECT', list[from], { slot_index: OPEN_SLOT });
    if (to > from) {
      for (let i = from + 1; i <= to; i += 1) {
        await updateDevice(track, 'EFFECT', byId(list, ids[i]), { slot_index: i - 1 });
      }
    } else {
      for (let i = from - 1; i >= to; i -= 1) {
        await updateDevice(track, 'EFFECT', byId(list, ids[i]), { slot_index: i + 1 });
      }
    }
    await updateDevice(track, 'EFFECT', byId(list, movedId), { slot_index: to });
    await loadDevices(track);
    refreshCells(track);
    showToast('Effect order changed');
  } catch (err) {
    warn(err);
    await loadDevices(track).catch(function () {});
    refreshCells(track);
  }
}

/** 목록에서 id 로 찾기 */
function byId(list, id) {
  return list.filter(function (x) { return x.track_plugin_id === id; })[0];
}
```

문법 풀이

식	뜻
`devices[track.track_id] = d`	값이 없으면 빈 자리를 먼저 만들어 둡니다
`tailSlot(list)`	지금 것들 뒤 빈 칸 번호를 찾습니다
`list.push(saved)`	서버가 돌려준 줄을 담아 둡니다
`window.confirm(...)`	떼기 전에 한 번 묻습니다
`list.splice(at, 1)`	담아 둔 목록에서 그 줄을 뺍니다
`ids.splice(from, 1)[0]`	옮길 줄의 번호를 꺼내고 목록에서 뺍니다
`ids.splice(to, 0, movedId)`	새 자리에 끼워 넣습니다
`await` 를 차례로	칸 번호가 겹치면 안 되므로 하나씩 저장합니다
`catch` 에서 다시 읽기	중간에 실패하면 서버 값으로 화면을 되돌립니다

4. 왜 이렇게

선택	이유
붙일 자리를 이어 붙임	새 카드가 늘 끝에 오는 것이 자연스럽습니다 (FR-263)
빈 칸 100 을 거쳐 옮김	`UQ(track_id, slot_index)` 때문에 자리를 맞바꿀 수 없습니다
하나씩 차례로 저장	두 줄이 같은 칸 번호를 잠깐이라도 함께 가질 수 없습니다
중복 규칙 방어	붙일 때도 `top + 1` 이 쓰이면 빈 칸을 찾습니다
떼기 전에 묻기	되돌릴 수 없는 동작입니다

5. 시험

시험 프로젝트에서 이펙트를 붙였다가(`Compressor → EQ3 → Gain`) 그 카드를 떼어 원래 2장으로 되돌렸습니다. 순서 바꾸기 시험에서 EQ3 를 앞으로 끌어 `slot_index` 가 1 에서 0 으로 바뀌고 Compressor 가 1 이 된 것을 `GET /tracks/4639/plugins` 로 확인했습니다.